

Smart Eyes

Meta Wearables (DAT) + Gemini Visual Inspection — MVP

Use-Case-Driven Client Build: Technical Documentation

Client: Swiss Safety Center AG

Author: Teodor Glisic

Lecturer: Prof. Dr. Yannick Suter

City, Date: Olten, 2026

Preface

This is client-facing technical documentation for the Smart Eyes MVP, one of two applications in the [smart_eyes_applications](#) repository. The MVP is the later, use-case-driven client build; it evolved from the earlier [SmartEyesV1-research](#) prototype (which carries the Phase 1–3 empirical evaluation framework). No source code is embedded here; each component is linked to its file in the repository. Setup, permission, and architecture explanations follow Meta's official [iOS integration guide](#) for the Wearables Device Access Toolkit (DAT).

Sanitized handoff copy

All developer-specific credentials (Meta ClientToken / MetaAppID, Apple Developer Team, Firebase configuration) have been removed or replaced with placeholders. See [SETUP.md](#) for exactly what must be supplied before the project will build.

How the MVP differs from V1

The MVP reuses the same glasses-to-Gemini pipeline validated in V1 but restructures the app around four task-specific use cases instead of three test phases, adds multi-turn follow-up questions about a single photo, grounds the workplace-safety use case in live Google Search results, adds bounded retry on transient model errors, and replaces V1's always-listening wake word with explicit tap-to-ask dictation. It also drops V1's simulator mock mode — the MVP always talks to real hardware and a real Firebase backend.

Table of Contents

Preface.....	II
Table of Contents.....	III
1 Project Overview	1
1.1 What the app does, in plain terms	1
1.2 Which Meta DAT SDK features it uses	1
2 Codebase Explanation	2
2.1 Project structure	2
2.2 Architecture at a glance.....	3
2.3 Key Files	4
2.4 How the Meta DAT SDK is implemented	5
2.5 How state is managed.....	5
2.6 Permissions and registration flow	6
2.7 How the Gemini integration works	6
2.8 What gets logged	7
2.9 Screenshots	9
3 Prerequisites & Setup.....	13
3.1 Tooling and hardware.....	13
3.2 Swift Package Manager dependencies	13
3.3 Info.plist and build-settings configuration.....	14
3.4 Credentials to supply (three places)	15
4 Step-by-Step Replication Guide	16
4.1 Part A — Prepare the glasses and accounts	16
4.2 Part B — Recreate and configure the app	16
4.3 Part C — Build, register, and verify	17
5 Extending the Project	18
5.1 Adding another Gemini model	18

5.2	Adding a new use case	19
5.3	Example: a complete use case entry	20
6	Known Rough Edges.....	21
7	Distribution & Signing Validity	22
	References.....	23

1 Project Overview

Smart Eyes (MVP) is an iPhone app that turns a pair of Meta smart glasses into a hands-free field-inspection assistant. A technician wearing the glasses looks at equipment — a pipe, a gauge, a machine, a work area — and the glasses' camera view is streamed to the paired iPhone. The technician takes a photo, asks a question about it (by voice or text), and Gemini answers in a couple of sentences suitable to be read back aloud through the glasses. Follow-up questions about the same photo stay in context, so it feels like a short conversation rather than a series of one-off lookups.

This is the **MVP / client build** of the bachelor-thesis project — the version restructured from the earlier research prototype into a practical, use-case-driven tool. Instead of abstract test phases, the technician picks the “lens” that matches the job at hand.

1.1 What the app does, in plain terms

- Pairs with Meta smart glasses and shows a live low-resolution preview of what the wearer sees.
- Captures a full-resolution photo on demand from the glasses.
- Lets the technician choose one of four use cases — Pipe Analysis, Gauge Reading, Anomaly Detection, or Workplace Safety (SUVA) — each tuned with its own expert prompt.
- Answers a spoken (tap-to-ask) or typed question about the photo, and can read the answer aloud.
- Keeps follow-up questions about the same photo in a running conversation, so the image doesn't need to be re-sent each time.
- Grounds the Workplace Safety use case in live Google Search results so it can cite current Swiss (SUVA) safety guidance and show its sources.
- Logs every trial and lets the technician rate accuracy and export the whole log as CSV.

1.2 Which Meta DAT SDK features it uses

The app is built on the [Meta Wearables Device Access Toolkit \(DAT\)](#) for iOS and uses the SDK's core capabilities: one-time configuration, app-to-glasses registration via the Meta AI companion app, continuous device discovery, device sessions, live camera streaming, and out-of-band high-resolution photo capture. Gemini access is handled separately through Firebase AI Logic, so no AI API key is ever stored in the app.

2 Codebase Explanation

The MVP is a SwiftUI iOS app organized into a small App shell, a set of manager singletons that each wrap one external capability, plain data models (including the cross-tab session state), and the SwiftUI views. All paths below link directly to the files in the repository.

2.1 Project structure

The repository folder is a single Xcode project with the Swift sources grouped by responsibility:

SmartEyesMVP/

└─ SmartEyesMVP/	App sources
└─ App/	
└─ SmartEyesApp.swift	Entry point, SDK + Firebase startup
└─ ContentView.swift	Tab shell (Setup · Inspection · Chat Log)
└─ Managers/	One singleton per external capability
└─ WearableManager.swift	Meta DAT SDK: register/stream/capture
└─ GeminiManager.swift	Gemini: multi-turn chat, grounding, retry
└─ VoiceQuestionManager.swift	Tap-to-ask dictation (no wake word)
└─ SpeechManager.swift	Text-to-speech playback
└─ TelemetryManager.swift	Trial logging + CSV export
└─ Models/	
└─ UseCaseCatalog.swift	The four use cases + safe-failure rule
└─ GeminiModelCatalog.swift	Selectable model list
└─ InspectionSessionState.swift	Cross-tab photo/chat state
└─ Views/	
└─ SetupView.swift	Registration, model, voice toggle
└─ InspectionView.swift	Live preview, use-case, capture
└─ ChatLogView.swift	Q&A, follow-ups, ratings, export
└─ ApiKeySettingsView.swift	Model-selection sheet
└─ Support/	
└─ Shared.swift	Shared UI helpers
└─ Theme.swift	Colors / styling
└─ Assets.xcassets/	App icon, accent color
└─ Info.plist	MWDAT config, URL scheme, bg modes
└─ GoogleService-Info.plist	(you supply – git-ignored)
└─ SmartEyesMVP.xcodeproj/	Xcode project + SPM resolution
└─ SETUP.md	Credentials you must supply
└─ README.md	In-repo technical reference

2.2 Architecture at a glance

The app follows a lightweight MVVM-style pattern. Each external capability — the glasses, Gemini, speech synthesis, voice dictation, telemetry — is isolated behind a single `@MainActor Observable-Object` singleton in the [Managers](#) folder. What sets the MVP apart from V1 is a dedicated cross-tab state object, [InspectionSessionState.swift](#), which carries the selected use case, the captured photo, whether a question has been asked about it, and the chat transcript between the Inspection and Chat Log tabs. Views observe the managers and this session object and re-render as their published properties change.

The entry point [SmartEyesApp.swift](#) configures Firebase and the Wearables SDK once at launch and routes the Meta registration callback URL to the glasses manager. [ContentView.swift](#) wires three tabs together — [SetupView.swift](#) (registration, model selection, voice toggle), [InspectionView.swift](#) (live preview, use-case selection, capture), and [ChatLogView.swift](#) (the question-and-answer conversation, ratings, and export).

2.3 Key Files

- [SmartEyesApp.swift](#) — App entry point. Configures Firebase and the Wearables SDK at startup and forwards the Meta registration callback URL to the glasses manager.
- [WearableManager.swift](#) — Wraps the Meta DAT SDK: registration, device discovery, session lifecycle, low-resolution camera streaming, and high-resolution photo capture. Unlike V1, this build has no simulator mock mode.
- [GeminiManager.swift](#) — Wraps Firebase AI Logic (Gemini). Runs multi-turn conversations over one photo, attaches Google Search grounding for use cases that need it, retries transient backend errors, and captures token telemetry.
- [VoiceQuestionManager.swift](#) — Tap-to-ask dictation. Captures a single spoken question (explicit start/stop, no wake word), which the technician can review and edit before sending.
- [SpeechManager.swift](#) — Local text-to-speech playback of answers, with markdown stripped for cleaner narration.
- [TelemetryManager.swift](#) — Persists every trial to local JSON, stores feedback (correct/incorrect, 1–5 stars), and exports the log as CSV.
- [UseCaseCatalog.swift](#) — The four use cases as data: each with a name, icon, system prompt, default question, and a Google-Search flag. Also holds the shared anti-hallucination guideline appended to every use case.
- [GeminiModelCatalog.swift](#) — The selectable Gemini/Gemma model list shown in settings and passed straight to Firebase AI Logic.
- [InspectionSessionState.swift](#) — The cross-tab state (selected use case, captured photo, chat history) shared between the Inspection and Chat Log tabs.
- [Info.plist](#) — Background modes, the smarteyesmpv URL scheme, external-accessory protocol, and the MWDAT SDK configuration block.

2.4 How the Meta DAT SDK is implemented

All glasses logic lives in [WearableManager.swift](#), which follows Meta's documented lifecycle end to end, matching the steps in the [iOS integration guide](#):

1. Configure — the SDK is configured once at launch.
2. Register — the user starts registration, which hands off to the Meta AI companion app; when that app calls back into the smarteyesmvp:// URL scheme, the entry point passes the URL to the SDK's URL handler. Registration and device state are observed through the SDK's registration and device streams.
3. Session — the manager waits for a device to be discovered, opens a session for that specific device using a SpecificDeviceSelector, and waits for the started state.
4. Stream — a camera stream is added at the lowest-bandwidth profile (low resolution, 2 FPS) to preserve Bluetooth Classic bandwidth for high-resolution photo capture. Frame, state, and error callbacks are wired to their publishers.
5. Capture — a JPEG photo capture triggers a high-resolution image over the photo-data publisher, independent of the low-resolution live preview; that captured image is what is sent to Gemini.

This is a deliberate downgrade rather than the starting design point: earlier iterations ran the live stream richer than 360×640 @ 2 FPS, but the glasses' Bluetooth Classic link has a fixed, small bandwidth budget shared between the continuous live preview and the on-demand high-resolution capture. When the preview was allowed to run any richer, it left too little headroom for the 1080×1440 capture to complete before the SDK's internal watchdog timeout fired, causing captures to silently fail or return corrupted or partial data. Deliberately starving the live preview down to the minimum useful frame rate was the fix — a choppier live view traded for every “Take Photo” action reliably returning a usable high-resolution image.

2.5 How state is managed

Each manager exposes observable, published properties describing its state; SwiftUI views bind to them, so the UI always reflects manager state. The distinctive piece in the MVP is [InspectionSessionState.swift](#), which owns the state that must survive moving between tabs: the selected use case, the captured photo, whether the current photo has already been asked about, and the chat transcript. Retaking a photo or switching use case clears the active conversation; purging telemetry resets the transcript. User preferences (selected model, voice on/off) persist through UserDefaults, and the trial log persists as JSON in the app's Documents directory via [TelemetryManager.swift](#).

2.6 Permissions and registration flow

Registration is a deep-link round trip: the user triggers it in the Setup tab; the DAT SDK opens the Meta AI companion app, where the user approves the connection and grants the glasses' camera/microphone permissions; the Meta AI app calls back through the `smarteyesmvp://` scheme declared in [Info.plist](#), and the app forwards that URL to the SDK to finish. Camera access to the glasses stream is requested through the SDK's permission API before streaming. On-device iOS permissions (microphone and speech recognition for dictation, photo-library add for saving captures) are requested at the point of use by [VoiceQuestionManager.swift](#), with their usage-description strings supplied through the project's build settings.

2.7 How the Gemini integration works

[GeminiManager.swift](#) talks to Gemini through Firebase AI Logic rather than a raw API key — the only key check is whether Firebase is configured, and the credential lives in `GoogleService-Info.plist`, never in app code. The MVP's analysis logic is richer than V1's in four ways:

- **Use-case prompts.** Each request uses the selected use case's system instruction from [UseCaseCatalog.swift](#), with a shared safe-failure guideline appended so the model explains why it can't answer rather than guessing.
- **Multi-turn conversation.** The first question about a photo opens a chat and sends the image and prompt together; follow-up questions about the same photo reuse that chat so the image isn't re-sent. Retaking the photo or switching use case starts a fresh conversation.
- **Google Search grounding.** The Workplace Safety (SUVA) use case attaches Gemini's Google Search tool so it can cite current Swiss safety guidance; when the model uses it, the response appends the source links, per Gemini's grounding usage terms.
- **Transient-error retry.** Only genuinely transient backend errors (HTTP 500/503, model overloaded) are retried — up to three attempts with backoff — while a 404 (retired model) or 429 (quota exhausted) fails immediately instead of hammering a dead endpoint.
- **Image pre-processing.** Before every request, the captured `UIImage` is run through `compressImageForAPI()` in [GeminiManager.swift](#), which always re-encodes it as JPEG at 0.95 quality, and would additionally downscale it if either dimension exceeded 1920px. In practice that downscale branch never fires: the two possible sources are the 1080×1440 high-res capture and the 360×640 live stream frame (used as a fallback), and both are already well under 1920px — so today this function only ever compresses, it never resizes. The

1920px cap looks like guardrail headroom for a future higher-resolution capture source rather than something the current pipeline exercises.

Every response's token usage (text, image, output, thinking, total) is captured for telemetry. If Firebase isn't configured, the manager returns a clear error string rather than a simulated answer — there is no canned fallback in this build.

2.8 What gets logged

TelemetryManager.swift stores one TrialRecord per analysis. The fields that matter for evaluation:

```
public struct TrialRecord: Codable, Identifiable {
    public let id: UUID
    public let timestamp: Date
    public let phase: String
    public let source: String
    public let captureMode: String
    public let imageWidth: Double
    public let imageHeight: Double
    public let latencySeconds: Double
    public let customPrompt: String
    public let rawResponseText: String
    public let parsedSeverity: String
    public let aiModel: String?
    public var isCorrect: Bool?
    public var rating1To5: Int?
    public var notes: String?
    public var inputTokens: Int?
    public var textInputTokens: Int?
    public var imageInputTokens: Int?
    public var outputTokens: Int?
    public var totalTokens: Int?
    public var thinkingTokens: Int?
}
```

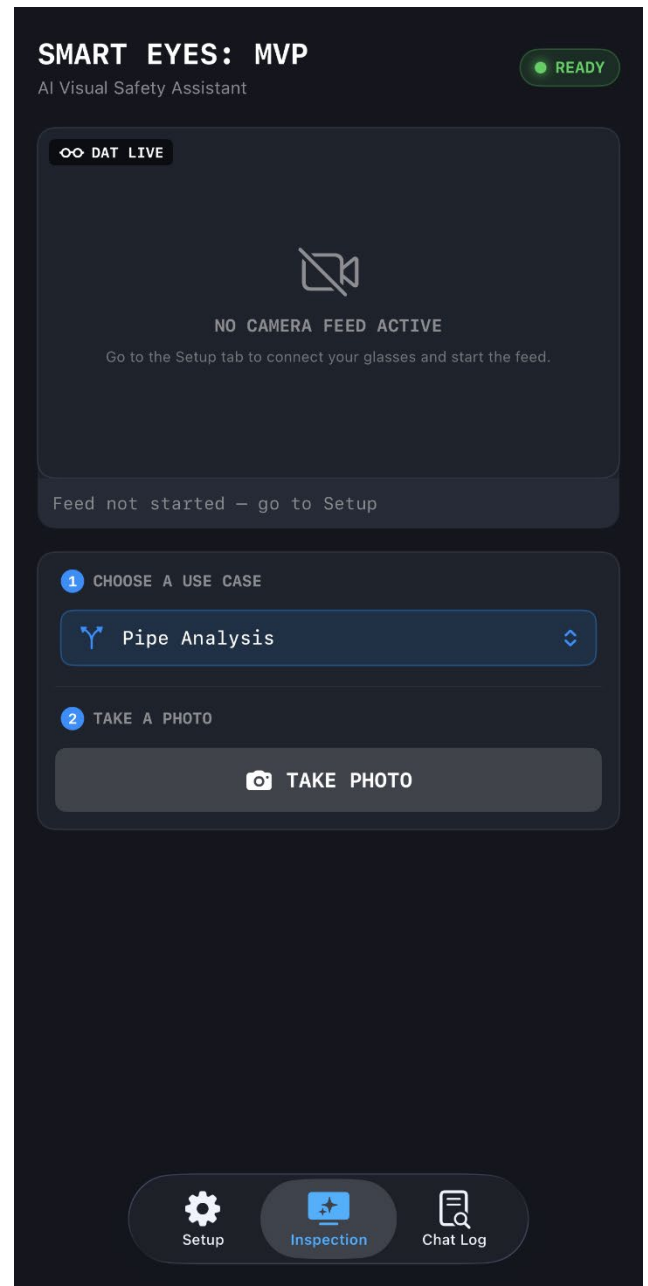
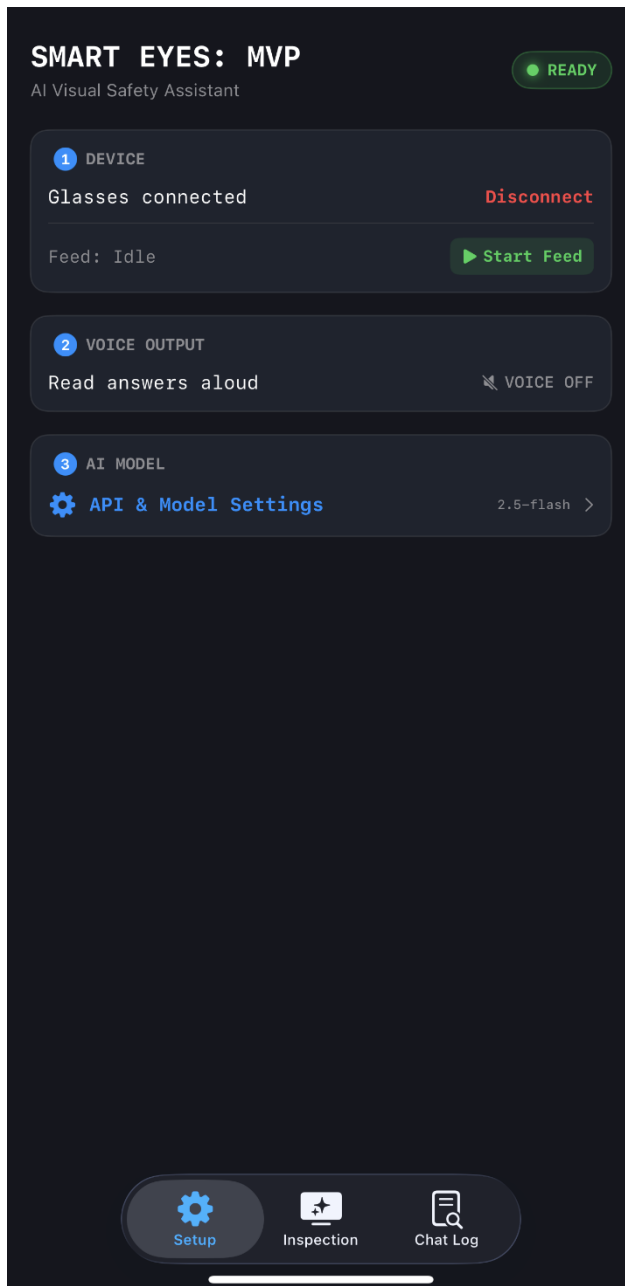
`exportCSV()` writes one double-semicolon-delimited row per record, with this header:

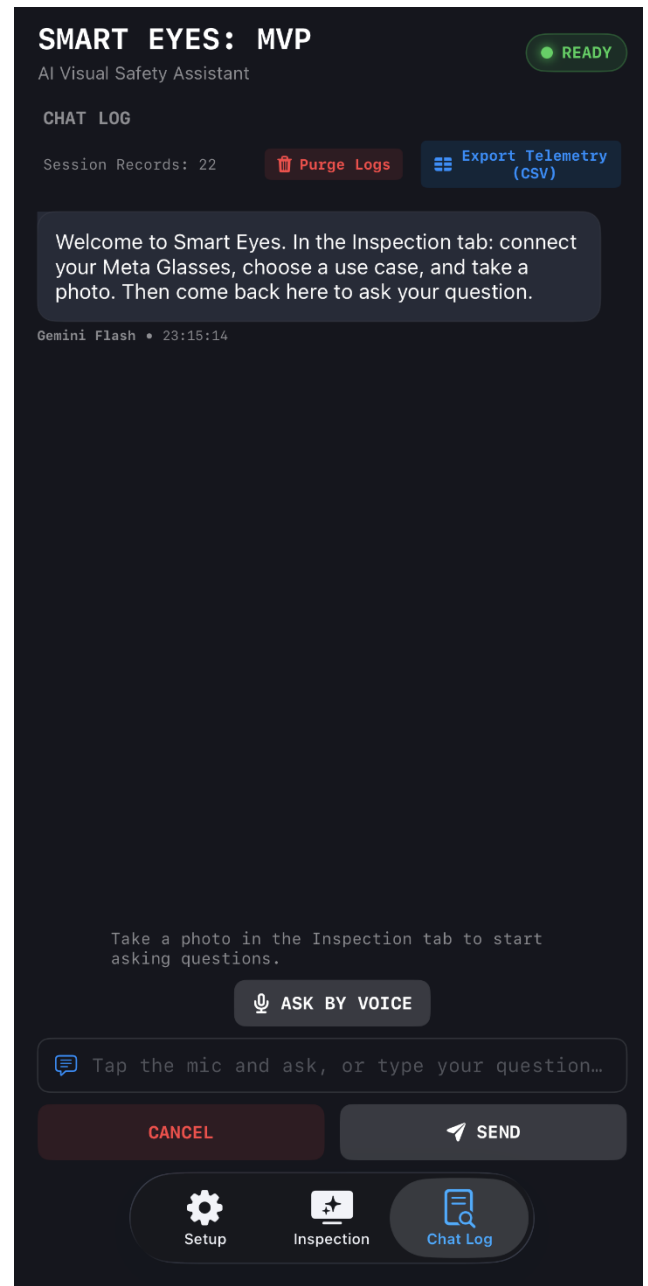
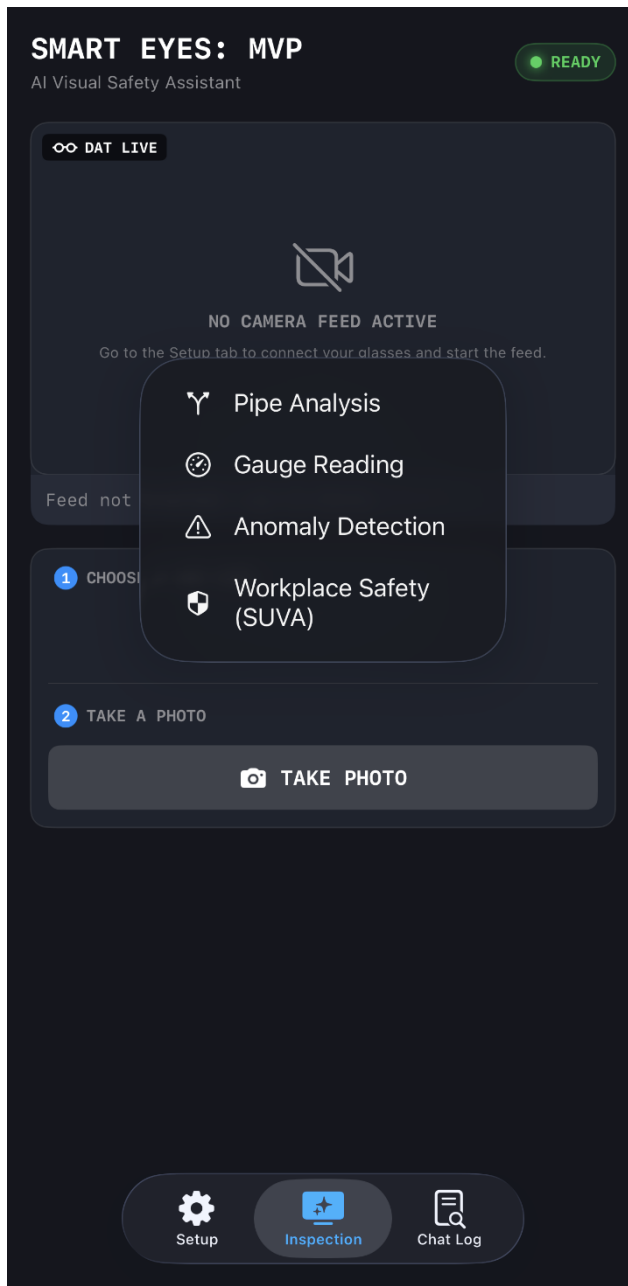
```
trial_id;;timestamp;;phase;;test_case;;source;;cap-  
ture_mode;;image_width;;image_height;;latency_seconds;;custom_prompt;;parsed_se-  
verity;;is_correct;;rating;;ai_model;;output;;response_status;;text_input_to-  
kens;;image_input_tokens;;total_input_tokens;;thinking_tokens;;output_tokens;;total_to-  
kens
```

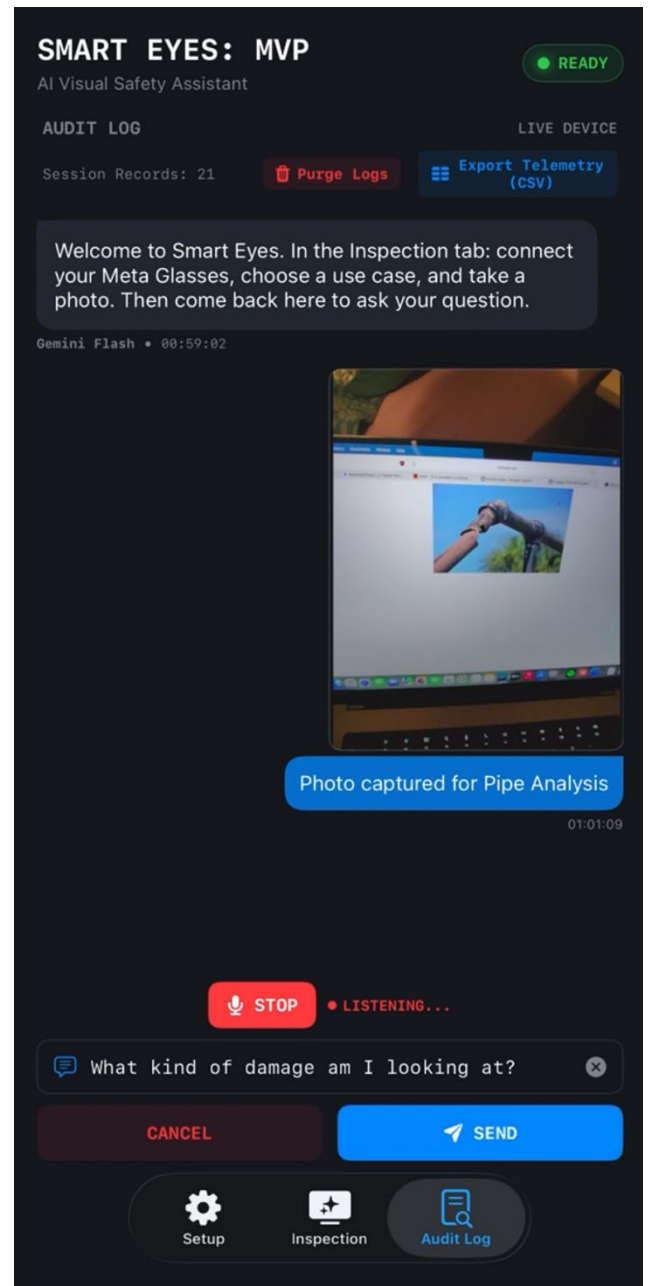
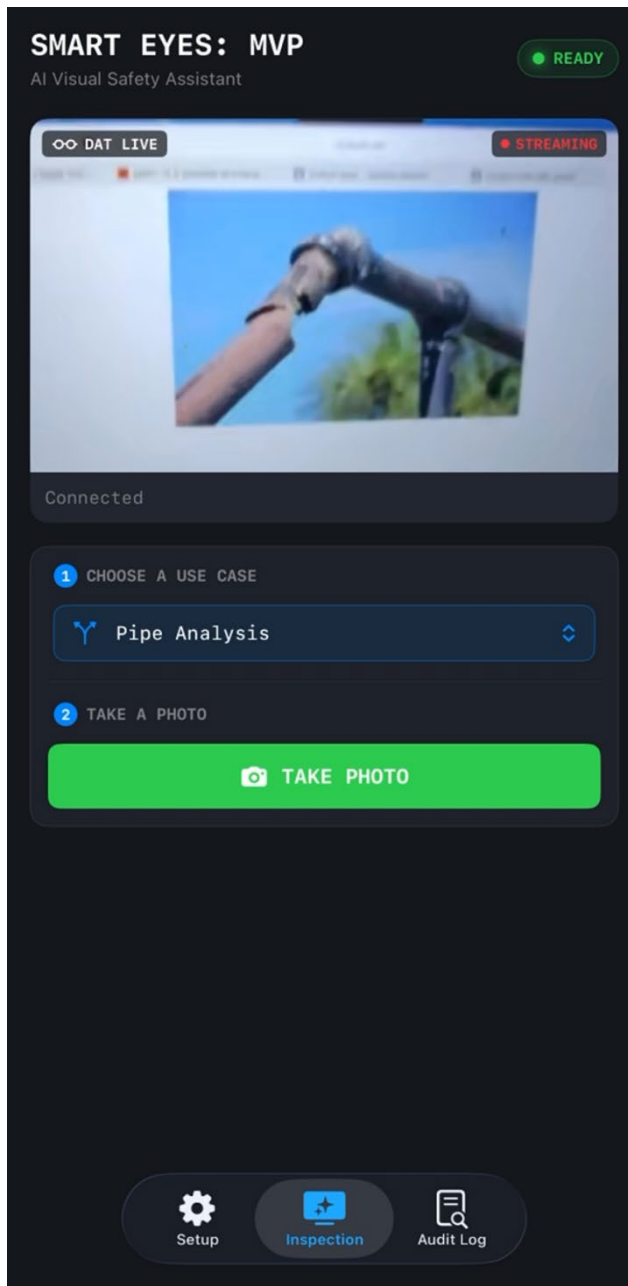
`test_case` always writes the literal placeholder string “Please enter test case”. That is intentional, not a bug: the tester manually overwrites this text per row in the exported spreadsheet with the actual test case label, so token usage can be analyzed per test case.

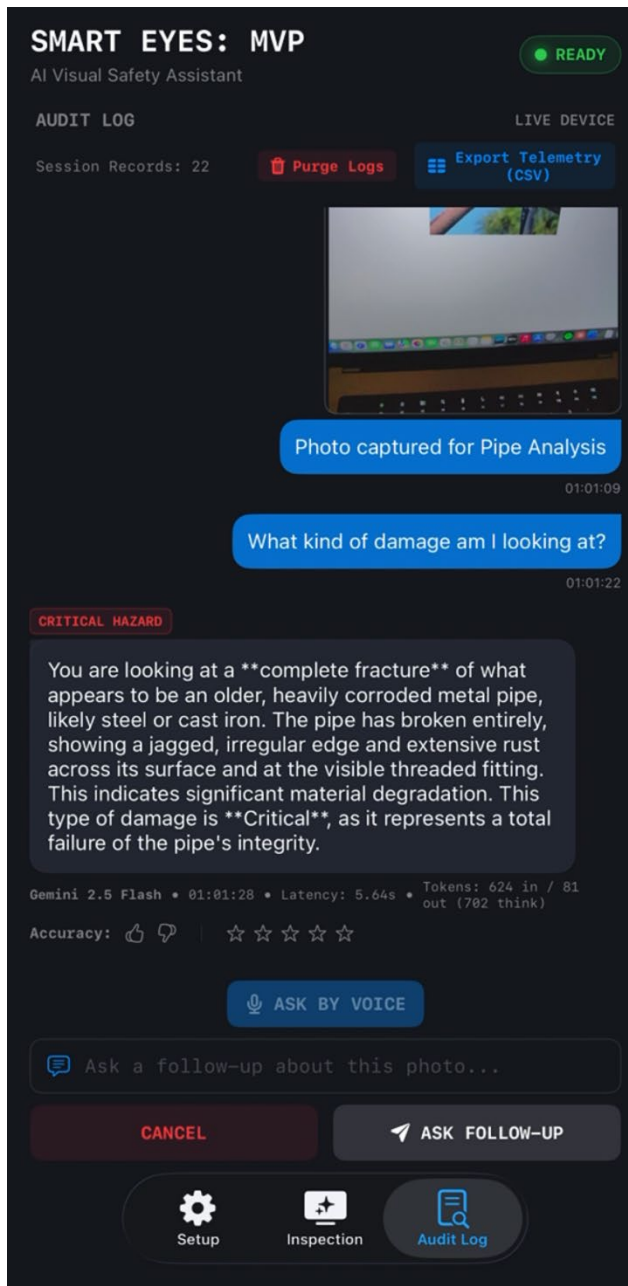
`parsedSeverity` is not a structured field from Gemini — it's a client-side keyword match over the model's free-text response (`ChatLogView.sendQuestion`): the lowercased answer is checked for “critical”, then “warning”, then “nominal”/“satisfactory”, in that order, defaulting to “N/A” if none match. It's only as reliable as the model's wording, since there's no enum or schema constraining the response.

2.9 Screenshots









3 Prerequisites & Setup

Everything below is required before the project will build and run. Because the MVP has no mock mode, a physical iPhone, paired glasses, and a configured Firebase project are all needed even for a first run.

3.1 Tooling and hardware

- **Xcode:** Version 26.5 (17F42). The project uses a very recent project format (objectVersion 77); Meta officially supports Xcode 14.0+ for the DAT SDK itself.
- **Host macOS:** 26.5.1 (25F80).
- **iOS deployment target:** 26.5 (all build configurations). A physical test device must run a matching iOS version.
- **Swift language version:** 5.0.
- **Test device:** A physical iPhone. The DAT SDK's camera-streaming path does not run in the Simulator, and this build has no mock fallback.
- **Glasses:** Ray-Ban Meta Wayfarer (Gen 2), paired to the Meta AI companion app with Developer Mode enabled. The SDK also supports Ray-Ban Meta Gen 1, Ray-Ban Meta Optics, and Meta Ray-Ban Display.
- **Accounts:** An Apple Developer account (for signing), a Meta Wearables Developer Center account (for the app credentials), and a Firebase project (for Gemini access).

3.2 Swift Package Manager dependencies

The project resolves two packages through Swift Package Manager — no manual download is needed:

- **Meta Wearables DAT SDK** — github.com/facebook/meta-wearables-dat-ios, resolved at version 0.8.0 (up-to-next-major rule, minimum 0.8.0), providing the MWDATCore and MWDATCamera products.
- **Firebase iOS SDK** — github.com/firebase/firebase-ios-sdk (resolved at 12.16.0), providing the FirebaseAI and FirebaseAILogic products used for Gemini access.

Both are public repositories added over HTTPS, so no additional GitHub credentials are needed to resolve them. When you clone this project, Xcode restores both automatically from the committed package resolution.

3.3 Info.plist and build-settings configuration

The DAT SDK is configured through the MWDAT dictionary and supporting keys in [Info.plist](#):

- **MWDAT › AppLinkURLScheme** = smarteyesmvp:// — the scheme the Meta AI app uses to call back after registration; it matches the CFBundleURLSchemes entry.
- **MWDAT › MetaAppID** = YOUR_META_APP_ID (placeholder) — replace with your Wearables Developer Center app ID.
- **MWDAT › ClientToken** = YOUR_META_CLIENT_TOKEN (placeholder) — replace with your Wearables Developer Center client token.
- **MWDAT › TeamID** = \$(DEVELOPMENT_TEAM) — populated automatically from your Apple signing team.
- **MWDAT › Analytics › OptOut** = true — this build opts out of Meta's SDK analytics collection.
- **CFBundleURLTypes** — declares the smarteyesmvp URL scheme for the registration call-back.
- **LSApplicationQueriesSchemes** = fb-viewapp, meta-view — lets the app reach the Meta companion apps.
- **UIBackgroundModes** = bluetooth-central, bluetooth-peripheral, external-accessory — keeps the glasses connection alive in the background.
- **UISupportedExternalAccessoryProtocols** = com.meta.ar.wearable — the External Accessory protocol the SDK communicates over.

The iOS permission usage-description strings (camera, microphone, speech recognition, local network, photo-library-add) are set in the target's build settings as INFOPLIST_KEY_* entries rather than in the Info.plist file. Note the MVP's MWDAT block does not include a DAMEnabled key; per Meta's guide the App Model defaults to enabled when the key is absent.

3.4 Credentials to supply (three places)

Full step-by-step instructions are in [SETUP.md](#). In summary:

1. **Firebase (Gemini access).** Create a Firebase project, enable AI Logic with the Gemini Developer API, add an iOS app using the bundle ID ch.glisic.SmartEyesMVP, download GoogleService-Info.plist, and place it next to the app source. This file is git-ignored and never committed.
2. **Meta Wearables Developer Center.** Register your own app to obtain a MetaAppID and ClientToken, then paste them over the two placeholders in Info.plist.
3. **Apple code signing.** The development team is blank in every build configuration; in Xcode, select the target → Signing & Capabilities → choose your Apple Developer Team. Xcode fills in the team ID, which also flows into the MWDAT TeamID key.

4 Step-by-Step Replication Guide

This recreates the project from scratch on a fresh machine, in order. Steps 1–4 prepare the glasses and accounts (following Meta's [Setup](#) guide); Steps 5–10 build and configure the app; Steps 11–13 run and verify it.

4.1 Part A — Prepare the glasses and accounts

1. **Update and pair the glasses.** Install the Meta AI companion app on the test iPhone, pair your Ray-Ban Meta Wayfarer (Gen 2) glasses, and confirm the Meta AI app and glasses firmware meet the DAT version requirements. To check the glasses version: Meta AI app → Devices tab → your device → gear icon → General → About → Version, and update if it is below the minimum.
2. **Enable Developer Mode in the Meta AI app.** Go to Settings → App Info and tap the App version number five times to reveal the Developer Mode toggle; enable it and confirm. Developer Mode lets your unpublished app register and interact with the glasses without review; registered apps appear under Meta AI settings → App connections → Developer mode apps.
3. **Installing the DAT onto the glasses:** After activating the developer mode, you will be prompted to install the latest DAT onto the glasses, which is mandatory, so that the application can communicate with the glasses and vice-versa.
4. **Register the app in the Wearables Developer Center.** Create a Wearables Developer Center account and register an app to obtain a MetaAppID and ClientToken for Step 7. (These identifiers are also used for app attestation; attestation is skipped in Developer Mode, but incorrect identifiers cause connection errors.)

4.2 Part B — Recreate and configure the app

5. **Get the project.** Clone the repository and open the MVP project.
https://github.com/teodorglisic/smart_eyes_applications
6. **Add the Swift Package dependencies.** If starting from scratch, in Xcode choose File → Add Package Dependencies..., add github.com/facebook/meta-wearables-dat-ios (version 0.8.0, up-to-next-major) and link its MWDATCore and MWDATCamera products, then add github.com/firebase/firebase-ios-sdk and link FirebaseAI and FirebaseAILogic. No GitHub credentials are required — both are public. When cloning the existing project, Xcode resolves these automatically.

7. **Configure Info.plist and permission strings.** Ensure the MWDAT block, the smarteyes MVP URL scheme, the background modes, and the External Accessory protocol are present (Section 3.3), and that the camera, microphone, speech-recognition, local-network, and photo-library-add usage descriptions are set in the target's build settings.
8. **Enter your Meta credentials.** Replace YOUR_META_APP_ID and YOUR_META_CLIENT_TOKEN in Info.plist with the values from Step 3.
9. **Set up Firebase for Gemini.** In the Firebase console, create/select a project, enable AI Logic → Gemini Developer API, add an iOS app with bundle ID ch.glisic.SmartEyesMVP, download GoogleService-Info.plist, and add it next to the app entry point. Firebase is already configured at startup in the entry point.
10. **Select your signing team.** Target → Signing & Capabilities → choose your Apple Developer Team so DEVELOPMENT_TEAM (and the MWDAT TeamID) is populated.

4.3 Part C — Build, register, and verify

11. **Build and run on the physical iPhone.** Select your device and run. (There is no Simulator path in this build.)
12. **Register the glasses.** In the Setup tab, with the glasses on and nearby, tap Register. You will be handed to the Meta AI app to approve the connection and grant camera/microphone access, then bounced back via the smarteyes MVP:// callback. Registration state should switch to registered and the device should appear as discovered.
13. **Run an inspection and confirm the pipeline.** In the Inspection tab, choose a use case (Pipe Analysis, Gauge Reading, Anomaly Detection, or Workplace Safety), take a photo, then ask a question by voice (tap-to-ask) or text. Gemini's answer should appear in the chat and, if voice output is on, be read aloud; follow-up questions about the same photo should stay in context. Confirm the Workplace Safety use case appends source links, and open the Chat Log tab to rate accuracy and export telemetry — verifying the full glasses → Gemini → report loop end to end.

5 Extending the Project

The selectable Gemini models and the use cases each live in a single catalog file, so extending either is a one-file change with no edits to the networking or UI code.

5.1 Adding another Gemini model

The model picker in the settings sheet is built directly from the list in [GeminiModelCatalog.swift](#). To offer a new model:

1. **Add one entry to the list.** In `GeminiModelCatalog.swift`, append a new model to the `GeminiModel.all` array with two fields: `id` — the exact model name Firebase AI Logic expects (for example `gemini-3.5-flash`) — and `displayName`, the label shown in the picker.
2. **Keep the id exact.** The id is persisted to `UserDefaults` and passed straight to Firebase AI Logic's `generativeModel(modelName:)`; an unknown name fails as a 404 at request time, not a compile error. Confirm the model is enabled for your Firebase/Gemini project first.
3. **Mind Google Search grounding.** The Workplace Safety use case attaches the Google Search tool, and only a subset of Gemini models support grounding. If you intend to use the new model with that use case, verify it supports grounding (see Gemini's [supported-models list](#)) before relying on it.

Nothing else needs to change: the picker in [ApiKeySettingsView.swift](#) iterates the catalog automatically, and the first entry is the default on a fresh install.

5.2 Adding a new use case

The four use cases are defined as data in [UseCaseCatalog.swift](#). Each is one `UseCase` value; the picker in the Inspection view and the grounding logic in [GeminiManager.swift](#) both just read off whichever use case is selected, so adding one touches only this file:

1. **Define the use case.** Create a new `UseCase` with an id (a unique short string), a name (shown in the picker), an icon (an SF Symbol name for the picker button), a `systemInstruction` (the expert prompt that defines the use case's behavior and reporting style), and a `defaultPrompt` (the question sent when the technician doesn't dictate their own).
2. **Reuse the safe-failure guideline.** End your system prompt by appending the shared `safeFailureGuideline` (as the existing use cases do) so the new use case inherits the same “say why you're unsure rather than guess” behavior.
3. **Opt into grounding if needed.** If the use case should consult live web results (as Workplace Safety does), pass `usesGoogleSearch: true`. Leaving it out defaults to false, and `GeminiManager` will attach the Google Search tool automatically only when it is true.
4. **Register it.** Add your new use case to the `UseCase.all` array so it appears in the Inspection tab's picker.

That is the entire change — no enum or switch edits anywhere else. (The sibling V1 build uses a similar catalog but calls these “phases,” carries simulator mock responses per entry, and has no per-entry grounding flag.)

5.3 Example: a complete use case entry

For reference, a complete, working use case — the shared `safeFailureGuideline` it references is defined once at the top of `UseCaseCatalog.swift` and appended by every entry:

```
static let pipeAnalysis = UseCase(  
    id: "pipe_analysis",  
    name: "Pipe Analysis",  
    icon: "arrow.triangle.branch",  
    systemInstruction: ""  
    You are "Smart Eyes", a visual assistant specialized in analyzing pipes and piping  
    systems for a field technician wearing smart glasses. The technician will show you a pipe  
    and ask a spoken question about it.  
    Guidelines:  
    1. Identify the pipe: Note the likely material (PVC, copper, steel, cast iron,  
    etc.), approximate diameter, and what it appears to carry (water, gas, drainage, process  
    fluid) if inferable from context, fittings, or labeling.  
    2. Answer the question directly: The technician's spoken question is your primary  
    task -- address it specifically and first, then add relevant supporting observations.  
    3. Condition & anomalies: Note visible corrosion, leaks, cracks, missing insula-  
    tion, improper support, or non-compliant fittings/joints.  
    4. Brevity: Keep responses concise (2-4 sentences) and suitable for being read  
    aloud through the glasses' speaker.  
    5. Severity: If you identify a genuine hazard or defect, state clearly whether  
    it is Critical, Warning, or Nominal.  
    6. \(safeFailureGuideline)  
    "",  
    defaultPrompt: "Analyze this pipe for its type, condition, and any visible issues."  
)
```

Appending a new `UseCase` to the `UseCase.all` array (in the same file) is the entire integration — no other file needs to change.

6 Known Rough Edges

A few known, low-severity issues are documented here for transparency rather than left implicit:

- Chat input toolbar polish. ChatLogView.swift carries a POLISH comment noting the chat text field's toolbar “Done” button placement feels slightly off — cosmetic only; the functional dismiss-timing behavior is already handled, so this is a nice-to-have revisit rather than a defect.
- Partial Google Search grounding compliance. For the Workplace Safety (SUVA) use case, source links are appended to the response as plain text when Gemini uses grounded search. Full compliance with Gemini's grounding usage terms also calls for rendering the “Google Search suggestions” widget from `groundingMetadata.searchEntryPoint.renderedContent` in a `WebView` — not wired up in this build.

7 Distribution & Signing Validity

Publishing to the App Store is not currently supported for DAT-based apps: Meta's SDK uses Apple's ExternalAccessory framework, which leads to App Store rejection under Apple's MFi program and privacy-manifest requirements (Meta, iOS integration guide). Meta states it plans to support submission in the future; until then, distribution is limited to development installs or Meta's release channels — so this project is installed directly from Xcode onto a physical device. How long that installed build keeps working depends on the Apple Developer account used to sign it.

- **Free Apple Developer account (a standard personal Apple ID):** the provisioning profile expires after 7 days. Once it lapses, launching the app on the device fails or crashes until you reconnect the device to Xcode and re-install / re-sign it.
- **Paid Apple Developer Program account (\$99/year):** a development or ad-hoc build installed via Xcode stays functional for up to 1 year, or until the paid membership expires — whichever comes first.

Notes and alternatives:

- **TestFlight:** with a paid account, builds distributed via TestFlight stay valid for 90 days.

For this project. Development used no paid subscription, so the installed build expires after 7 days — but it can simply be reinstalled / re-loaded from Xcode manually to reset the 7-day window.

References

- [Meta Wearables DAT — iOS integration guide](#)
- [Meta Wearables DAT — Setup / getting started](#)
- [Gemini grounding — supported models](#)
- [facebook/meta-wearables-dat-ios \(SDK source\)](#)
- [firebase/firebase-ios-sdk](#)
- [Firebase AI Logic documentation](#)
- [Project repository](#)